KIRAN ACADEMY

JAVA & PYTHON PROGRAMMING TASK

TASK 04: FLIGHT BOOKING CONSOLE APPLICATION — OOP

Submission Email	Email Subject	Task Date
9011@thekiranacademy.com	Task 04 – Flight Booking OOP – 23/10/2026	23 October 2026
Deadline	**Submission**	**Language**
23 October 2026, 11:59 PM	GitHub repository URL only	Java or Python

1. Problem Statement

Build a beginner-friendly, console-based **Flight Booking Application** using Object-Oriented Programming (OOP). Students who prefer Java can implement it in Java, and students who prefer Python can implement the same requirements in Python. The application should display available flights, allow a user to select a flight and enter passenger details, calculate the ticket fare, confirm a booking, and view the booking summary. Use classes and objects instead of writing the entire program inside one main function.

2. Required Program Flow

Step	Action	What it must do
1	Main menu	Display: 1. View Flights 2. Book Flight 3. View Booking 4. Exit. Repeat until Exit is selected.
2	View flights	Display at least 3 predefined flights with flight code, airline, source, destination, available seats, and base fare.
3	Book a flight	Ask for a flight code, passenger name, age, and number of tickets. Validate all inputs before confirming the booking.
4	Calculate fare	Calculate total fare based on the selected flight and passenger category. Show base fare, any discount, and final amount.
5	Booking summary	Generate a booking ID and display passenger details, flight details, ticket count, and total fare. Store the confirmed booking in memory.
6	View booking	Display the most recent confirmed booking. If no booking exists, show a helpful message.
7	Exit / invalid input	Exit with a goodbye message. Invalid choices or booking details must show an error without changing seat counts or creating a booking.

3. OOP Requirements — Mandatory

Your solution must demonstrate the following OOP concepts. You may choose suitable class and method names, but the responsibilities must be clear.

Concept	Requirement
Class and Object	Create classes such as **Flight**, **Passenger**, **Booking**, and **FlightBookingSystem**. Create objects to represent flights, passengers, and bookings.
Encapsulation	Keep related data and operations together in classes. Use private fields/getters/setters in Java or controlled access through methods/properties in Python where appropriate. Do not modify flight seat counts randomly from the menu code.
Inheritance	Create a base class **Passenger** and at least two passenger types, such as **RegularPassenger** and **StudentPassenger**. Each type should provide its own discount percentage.

Concept	Requirement
Polymorphism	Call the same fare-discount method on different passenger objects. The correct discount should be applied according to the actual passenger type.
Abstraction	Define a common fare/discount contract. In Java, use an abstract class or interface; in Python, use an abstract base class or a clearly defined common method overridden by subclasses.
Methods	Use separate methods for displaying flights, finding a flight by code, validating booking details, calculating fare, confirming a booking, and displaying the summary.

4. Flight Data and Booking Rules

• **Predefined flights:** Create at least 3 flights, for example Pune → Delhi, Mumbai → Bengaluru, and Pune → Hyderabad. Use your own flight codes, seat counts, and base fares.

• **Passenger discounts:** Regular passenger = 0%; student passenger = 10%. Apply the discount to the fare before calculating the final total.

• **Fare formula:** Subtotal = base fare × number of tickets. Discount = subtotal × passenger discount percentage. Final total = subtotal – discount.

• **Validation:** Flight code must exist; passenger name cannot be blank; age must be greater than 0; ticket count must be greater than 0.

• **Seat availability:** A booking is allowed only when the requested ticket count is less than or equal to the selected flight's available seats.

• **Booking confirmation:** Decrease available seats and save the booking only after every validation succeeds. Invalid or unsuccessful bookings must not change seat counts.

• **Data storage:** Keep flights and bookings in program memory. No database, API, GUI, file storage, or external library is required.

• **Menu loop:** Use a loop (for example, `while (true)` in Java or `while True` in Python) and exit cleanly.

5. Sample Input and Expected Output

Sample Scenario A — Successful student booking

Illustrative input sequence: 2 (Book Flight) → PN101 → Asha Patil → 21 → Student → 2 tickets → 3 (View Booking) → 4 (Exit)

```
Welcome to Kiran Air Booking
1. View Flights
2. Book Flight
3. View Booking
4. Exit
Enter choice: 2
Enter flight code: PN101
Enter passenger name: Asha Patil
Enter age: 21
Passenger type (Regular/Student): Student
Enter number of tickets: 2

Booking confirmed!
Booking ID: BK1001
Flight: PN101 | Pune to Delhi
Passenger: Asha Patil
Passenger type: Student
Tickets: 2
Base fare per ticket: ■4,000
Subtotal: ■8,000
Student discount (10%): ■800
Final amount: ■7,200
Enter choice: 4
Thank you for using Kiran Air Booking!
```

Sample Scenario B — Not enough seats

Input: Select flight PN101 → request 50 tickets when only 20 seats are available.

```
Booking failed: Not enough seats available.
No booking was created. Available seats remain unchanged.
```

Your exact wording and menu layout may differ. The OOP structure, fare calculations, validation, and seat updates must be correct.

6. Mandatory Submission Instructions

• Use Java or Python. Choose one language; submitting both is optional.

• Use classes and objects. A solution written entirely as standalone functions without the required OOP design does not meet this task.

• Include your full name in the email body.

• Upload source code to a GitHub repository and submit the repository URL only. Do not attach source-code files to the email.

• Email: **9011@thekiranacademy.com**

• Subject: **Task 04 – Flight Booking OOP – 23/10/2026**

• Deadline: **23 October 2026, 11:59 PM**. Make sure your repository is accessible.

7. Email Body Example

> Hello, I am [Your Full Name]. I have completed Task 04 – Flight Booking OOP in [Java/Python].
> GitHub Repository: [Paste your repository URL]

Challenge: Keep your code readable, give each class one clear responsibility, and test both successful and failed bookings before submitting.